EXPERIMENT 4

D15A-57

Aim: Implement K-Nearest Neighbors (KNN) and evaluate model performance

Theory:

1. Dataset Source
Source: Kaggle – Iris Species Dataset
Link: https://www.kaggle.com/datasets/uciml/iris

2. Dataset Description

The Iris dataset consists of 150 instances and 4 numerical features:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

The target variable is "Species", which has three classes:

- Iris-setosa
- Iris-versicolor
- Iris-virginica

It is a multiclass classification problem with no missing values.

3. Mathematical Formulation of the Algorithm (KNN)

K-Nearest Neighbors is based on distance calculation.

Let a training dataset be:

$D = \{(x1, y1), (x2, y2), ..., (xn, yn)\}$

Where:

- xi = feature vector
- yi = target label

For a new data point x, the distance is calculated using Euclidean Distance:

$d(x, xi) = \sqrt{\Sigma (xj - xij)^2}$

The algorithm selects the K nearest neighbors based on smallest distances.

For classification:
Predicted class = Majority vote of K neighbors

For regression:
Predicted value = Mean of K neighbors

4. Algorithm Limitations

1. Computationally expensive for large datasets.
2. Requires feature scaling.
3. Sensitive to noise and irrelevant features.
4. Performance depends heavily on the value of K.
5. Not suitable for high-dimensional data.

5. Methodology / Workflow

The experiment follows these steps:

1. Data Collection
   Load dataset from source.
2. Data Preprocessing
   - Handle missing values
   - Encode categorical variables
   - Scale numerical features
3. Feature Selection
   Select independent variables (X) and target variable (Y).
4. Train-Test Split
   Split dataset into training and testing sets.
5. Model Training
   Apply KNN algorithm with chosen K value.
6. Prediction
   Predict output for test data.
7. Performance Evaluation
   Calculate accuracy (classification) or RMSE/R^2 (regression).

6. Performance Analysis

For Classification:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

For Regression:

- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score

Interpretation:

Higher accuracy or R^2 indicates better model performance. Moderate values suggest improvement can be made through better feature selection or tuning.

7. Hyperparameter Tuning

The main hyperparameter in KNN is:

- K (number of neighbors)

Procedure:

- Test multiple K values (1 to 15)
- Compare performance metrics
- Select the K with best accuracy or lowest error

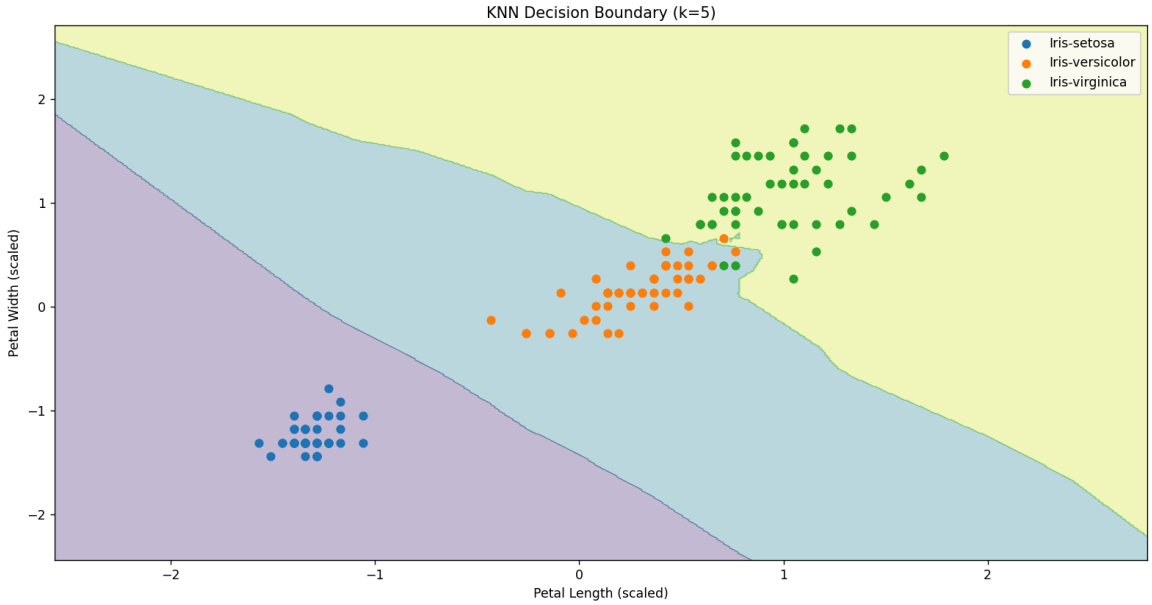
Hyperparameter tuning improves model stability and prediction performance.

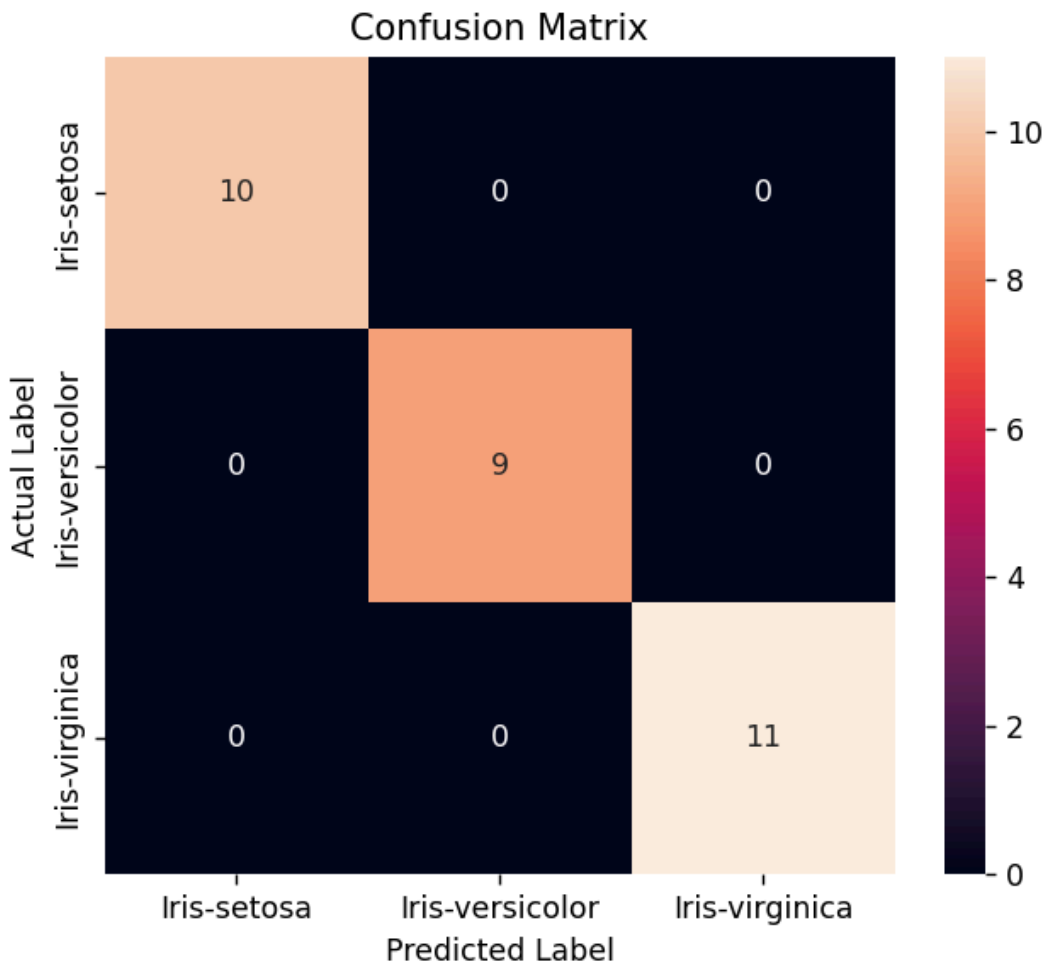
Code:

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
from matplotlib.colors import ListedColormap

df = pd.read_csv("Iris.csv")

if 'Id' in df.columns:
    df = df.drop(['Id'], axis=1)

X = df[['PetalLengthCm', 'PetalWidthCm']].values
y = df['Species'].values
le = LabelEncoder()
y_encoded = le.fit_transform(y)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_encoded, test_size=0.2, random_state=42
)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=le.classes_))
```

```python
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm,
            annot=True,
            fmt='d',
            xticklabels=le.classes_,
            yticklabels=le.classes_)

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix")
plt.show()

x_min, x_max = X_scaled[:, 0].min() - 1, X_scaled[:, 0].max() + 1
y_min, y_max = X_scaled[:, 1].min() - 1, X_scaled[:, 1].max() + 1

xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01),
                     np.arange(y_min, y_max, 0.01))

Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.figure(figsize=(8,6))
cmap_light = ListedColormap(['#FFCCCC', '#CCFFCC', '#CCCCFF'])
plt.contourf(xx, yy, Z, alpha=0.4, cmap=cmap_light)

colors = ['red', 'green', 'blue']
for i, color in enumerate(colors):
    plt.scatter(X_scaled[y_encoded == i, 0],
                X_scaled[y_encoded == i, 1],
                c=color,
                label=le.classes_[i])
plt.xlabel("Petal Length (scaled)")
plt.ylabel("Petal Width (scaled)")
plt.title("KNN Decision Boundary (k=5)")
plt.legend()
plt.show()
```

Output:


KNN Decision Boundary (k=5)
Iris-setosa
Iris-versicolor
Iris-virginica
Petal Width (scaled)
Petal Length (scaled)
2
1
0
−1
−2
−2
−1
0
1
2




```
Classification Report:
                 precision    recall  f1-score

    Iris-setosa       1.00      1.00      1.00
Iris-versicolor       1.00      1.00      1.00
 Iris-virginica       1.00      1.00      1.00

       accuracy                           1.00
      macro avg       1.00      1.00      1.00
   weighted avg       1.00      1.00      1.00
```

Conclusion:

K-Nearest Neighbors is a simple yet powerful algorithm that makes predictions based on similarity. It is effective for both classification and regression problems. Although computationally expensive for large datasets, it performs well when data is properly scaled and relevant features are selected.